TinyML @Penn

Penn Engineering

Ordering for the presentation demo

Serial Number	Team Members	Project Title	Slot	Time duration(mins)
1	Brandon and Maya	Live Pitch Correction	4:00PM	10
2	Maxwell and Eric	Gains	4:10 PM	10
3	Nathan and Kristian	Connect 4 AI Player	4:20 PM	10
4	Michael and Vidhu	Swingsense	4:30 PM	10
5	Emma and Roshni	Page Flipper	4:40 PM	10
6	Emil and Victor	TinyML Racer	4:50 PM	10
7	Oscar and Justin	Card Counter	5:00 PM	10
8	Aaron and Dailing	MoodSync	5:10 PM	10
9	James and Tim	OhmSweetOhm	5:20 PM	10
10	Grace and Sarah	BabyLover3600	5:30 PM	10
11	Vivasvat and Jeffrey	Orbitone (prev. AirDrums)	5:40 PM	10
12	Jackson and Jefferson	PawtWhole	5:50 PM	10
13	Jevin and Yash	Distracted Driving	6:00 PM	10
14	Jeremy and Dhruva	ServeSense	6:10 PM	10

Ordering for the presentation demo

Serial Number	Team Members	Project Title	Slot	Time duration(mins)
15	Amanda and Sofia	ClearChord	6:20 PM	10
16	Evelyn and Karina	DailyDose	6:30 PM	10
17	Kieran and Mide	ASL Detection	6:40 PM	10
18	Leo		6:50 PM	10
19	Shishir and Luke	Sink Secretary	7:00 PM	10

Example project presentation

- ➔ Use slides 5-13 as a template
- ➔ Put your slides after this example
- ➔ Don’t overwrite others’ slides

Penn Engineering

LaTeXeR

Handwriting to LaTeX Conversion

Bo Sun

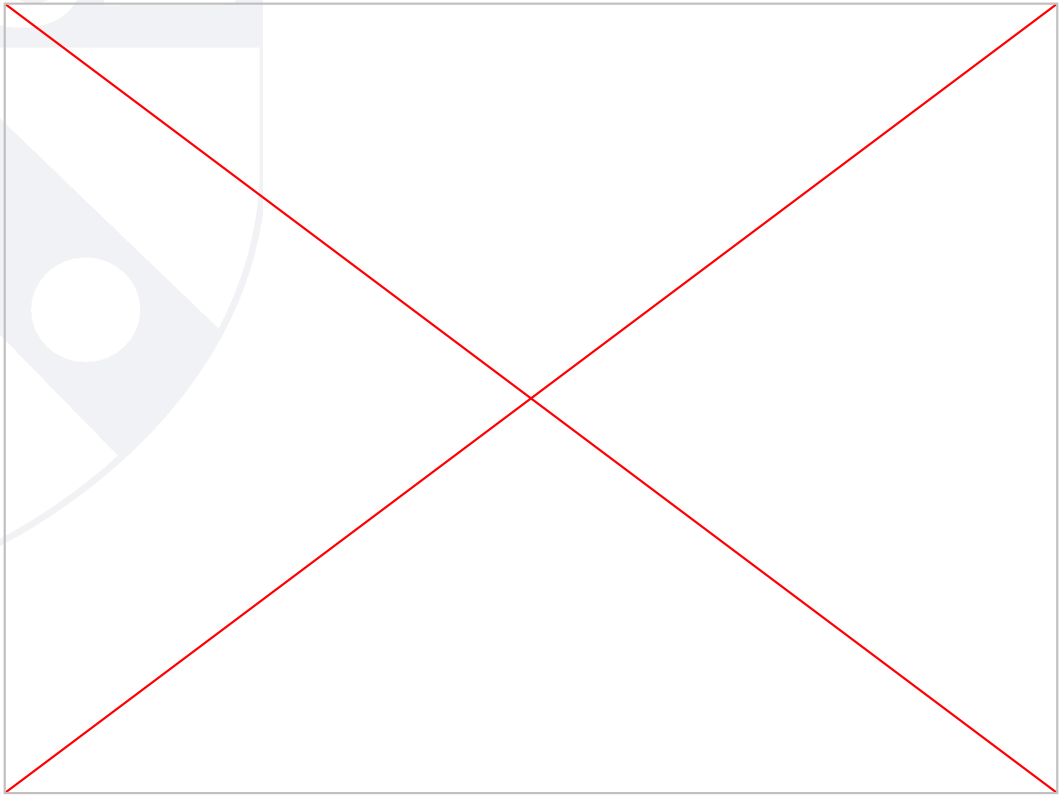
(previous years were mostly solo student projects)

Penn Engineering

Handwriting to LaTeX

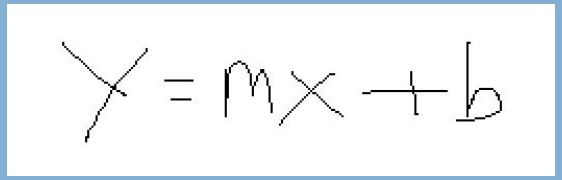
- Why make this app?
  - Love math, hate LaTeX
  - ~~Too lazy for CIS 1600 homework~~
- Challenge: problem involves many steps
  - Parse input for individual characters
  - Model predicts symbols
  - Postprocess into LaTeX equation

Demo

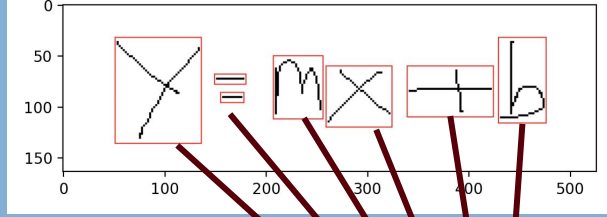


System Pipeline

Input Image



Parse Image



Model

Postprocessing

```
def predict(image, model):
    symbol_bounds = image_segmentation.segment_image(image)
    image = image_segmentation.bw_to_rgb(image_segmentation.rgb_to_bw(image))
    latex_string = ''

    for box in symbol_bounds:
        symbol = image[box['min_y']:box['max_y'], box['min_x']:box['max_x']]
        symbol = image_segmentation.resize_symbol(symbol, 45)
        symbol = symbol / 255.

        predict_scores = model.predict(tf.reshape(symbol, [1, 45, 45, 3]), verbose = 0)
        max_predict = np.argmax(predict_scores)
        predict_label = symbol_recognition.labels[max_predict]

        latex_string = latex_string + predict_label

    latex_string = latex_string.replace('--', '=')
    latex_string = latex_string.replace('->', '\\geq')
    latex_string = latex_string.replace('-<', '\\leq')
    latex_string = latex_string.replace('%', '\\')
    return latex_string
```

Output

```
1/1 [==============================] - 0s 102ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 9ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 8ms/step
prediction: y=mx+b || label: y=mx+b
```

Penn Engineering

Model Architecture

Input Symbol



Conv2D(8, (3,3), activation='relu')

MaxPooling2D(2,2)

Conv2D(16, (3,3), activation='relu')

MaxPooling2D(2,2)

Conv2D(32, (3,3), activation='relu')

MaxPooling2D(2,2)

Flatten()

Dense(128, activation='relu')

Dense(128, activation='relu')

Dense(68, activation='softmax')

Output

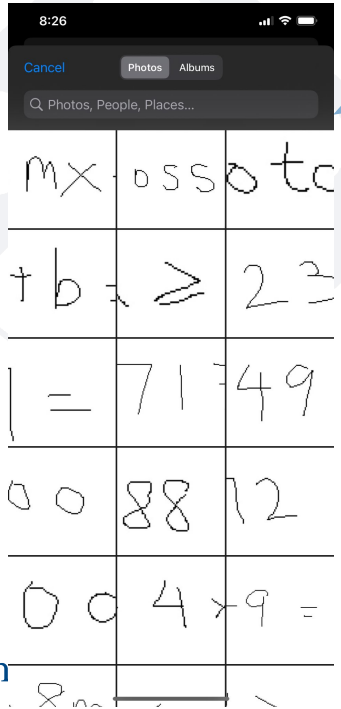
\Delta

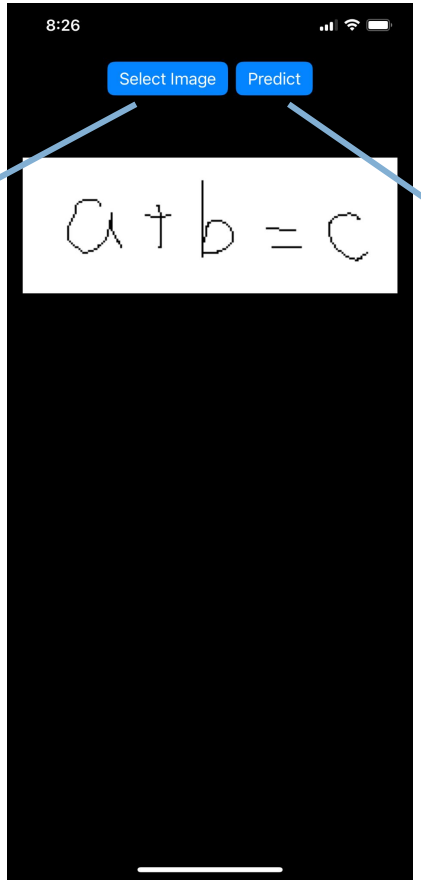
Training and Testing Results

- Accuracy
  - Individual symbols: all accuracies ~98%
  - Performs worse on equations after image parsing
- Difficulties
  - Letter ‘x’ vs multiplication symbol ‘\times’
  - Letter ‘i’ vs number ‘1’ vs vertical bar ‘\vert’
  - Letter ‘o’ vs number ‘0’

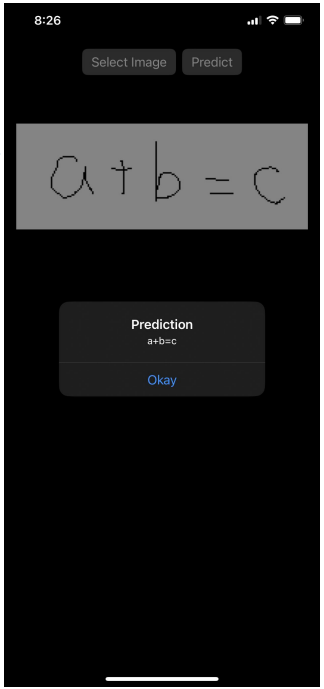
iOS Embedding

Brings up small subview containing a photo selector





Invokes model, gives a pop-up containing prediction



Penn

More About the App

- Languages
  - Python
  - Swift
  - Objective C/C++
- External Dataset
  - Handwritten Math Symbols Dataset (100,000+ images)
- External Libraries
  - **OpenCV:** for image manipulation, such as thresholding, contour detection, and bounding boxes
  - **CoreML:** for integration of Tensorflow with iOS
- Porting from initial Python code into Objective C/C++ and Swift

Future Improvement

- Improved accuracy
- Improved user interface
- More complex equations (fractions, square roots, etc.)
- Real-time LaTeX compilation
- Save past results
- Live camera capture

Copy the template - last 9 slides

Penn Engineering

Start your slides after this…

Penn Engineering

Connect 4 AI Player

Connect 4 ML Vision and Next Move Prediction

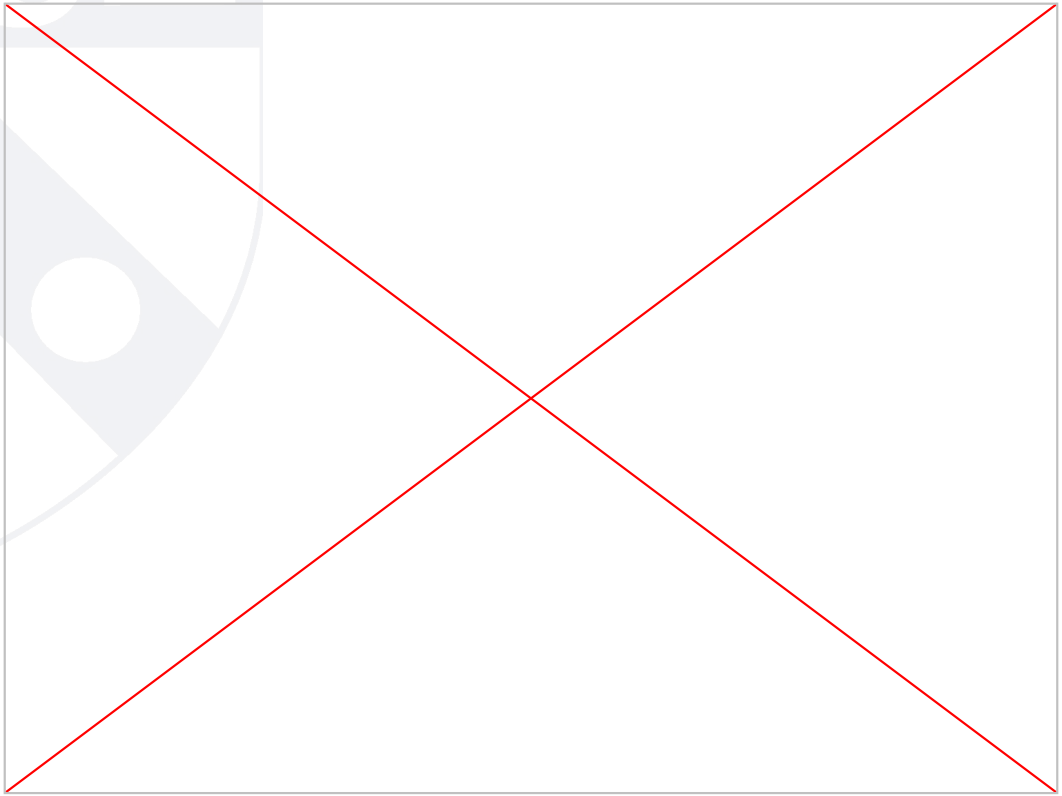
Nathan Zhang and Kristian Prifti

Penn Engineering

Handwriting to LaTeX

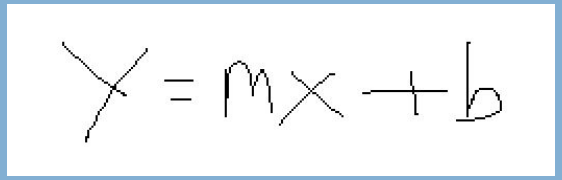
- Why make this app?
  - Love math, hate LaTeX
  - ~~Too lazy for CIS 1600 homework~~
- Challenge: problem involves many steps
  - Parse input for individual characters
  - Model predicts symbols
  - Postprocess into LaTeX equation

Demo

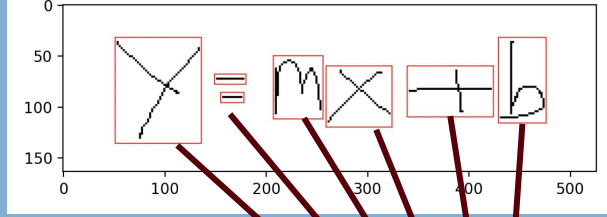


System Pipeline

Input Image



Parse Image



Model

Postprocessing

```
def predict(image, model):
    symbol_bounds = image_segmentation.segment_image(image)
    image = image_segmentation.bw_to_rgb(image_segmentation.rgb_to_bw(image))
    latex_string = ''

    for box in symbol_bounds:
        symbol = image[box['min_y']:box['max_y'], box['min_x']:box['max_x']]
        symbol = image_segmentation.resize_symbol(symbol, 45)
        symbol = symbol / 255.

        predict_scores = model.predict(tf.reshape(symbol, [1, 45, 45, 3]), verbose = 0)
        max_predict = np.argmax(predict_scores)
        predict_label = symbol_recognition.labels[max_predict]

        latex_string = latex_string + predict_label

    latex_string = latex_string.replace('--', '=')
    latex_string = latex_string.replace('->', '\\geq')
    latex_string = latex_string.replace('-<', '\\leq')
    latex_string = latex_string.replace('%', '\\')
    return latex_string
```

Output

```
1/1 [==============================] - 0s 102ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 10ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 9ms/step
1/1 [==============================] - 0s 8ms/step
1/1 [==============================] - 0s 8ms/step
prediction: y=mx+b || label: y=mx+b
```

Penn Engineering

Model Architecture

Input Symbol



Conv2D(8, (3,3), activation='relu')

MaxPooling2D(2,2)

Conv2D(16, (3,3), activation='relu')

MaxPooling2D(2,2)

Conv2D(32, (3,3), activation='relu')

MaxPooling2D(2,2)

Flatten()

Dense(128, activation='relu')

Dense(128, activation='relu')

Dense(68, activation='softmax')

Output

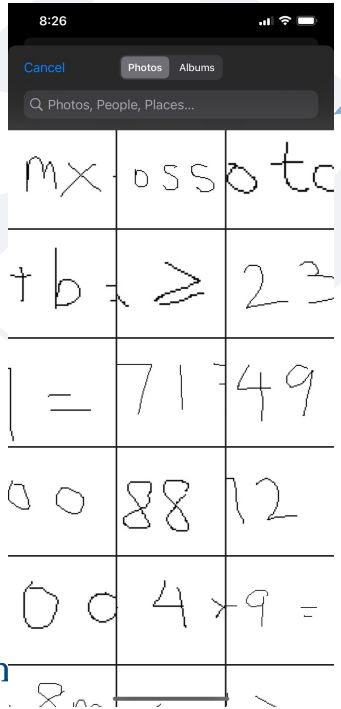
\Delta

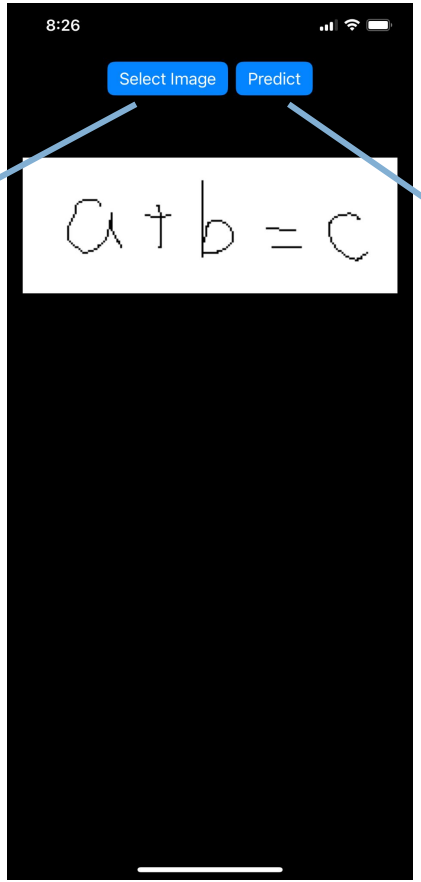
Training and Testing Results

- Accuracy
  - Individual symbols: all accuracies ~98%
  - Performs worse on equations after image parsing
- Difficulties
  - Letter ‘x’ vs multiplication symbol ‘\times’
  - Letter ‘i’ vs number ‘1’ vs vertical bar ‘\vert’
  - Letter ‘o’ vs number ‘0’

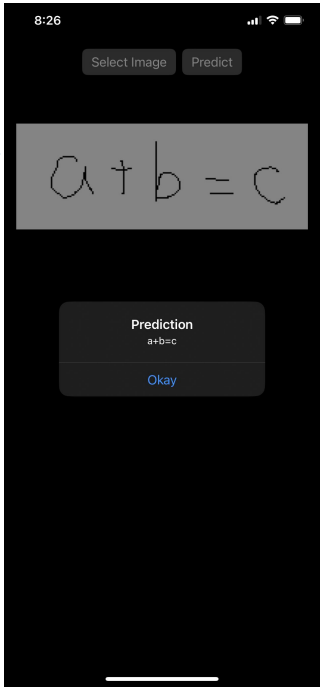
iOS Embedding

Brings up small subview containing a photo selector





Invokes model, gives a pop-up containing prediction



Penn

More About the App - UNFINISHED

- Languages
  - Python
  - Swift
  - Objective C/C++
- External Dataset
  - Handwritten Math Symbols Dataset (100,000+ images)
- External Libraries
  - **OpenCV:** for image manipulation, such as thresholding, contour detection, and bounding boxes
  - **CoreML:** for integration of Tensorflow with iOS
- Porting from initial Python code into Objective C/C++ and Swift

Future Improvement - UNFINISHED

- Improved accuracy
- Improved user interface
- More complex equations (fractions, square roots, etc.)
- Real-time LaTeX compilation
- Save past results
- Live camera capture